26.2 Cron 作业执行

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~210k tokens，输出 ~6k tokens（本节）

上一节介绍了 Cron 系统的调度设计。这一节深入作业的实际执行：如何区分主会话作业和隔离作业、投递计划如何决定消息去向、运行日志如何记录、以及作业存储的持久化机制。

26.2.1 隔离 Agent 执行（`src/cron/isolated-agent.ts`）

两种执行模式

Cron 作业有两种执行模式，由 `sessionTarget` 字段决定：

模式	`sessionTarget`	负载类型	工作方式
主会话	`"main"`	`systemEvent`	向主会话注入系统事件文本，触发心跳
隔离会话	`"isolated"`	`agentTurn`	启动独立的 Agent 会话执行任务

```
// src/cron/service/timer.ts — executeJobCore（简化版）

async function executeJobCore(state, job) {
  // 模式一：主会话执行
  if (job.sessionTarget === "main") {
    const text = resolveJobPayloadTextForMain(job);
    if (!text) return { status: "skipped", error: "requires non-empty text" };

    // 注入系统事件
    state.deps.enqueueSystemEvent(text, { agentId: job.agentId });

    // 唤醒 Agent 处理
    if (job.wakeMode === "now" && state.deps.runHeartbeatOnce) {
      // 同步模式：等待心跳完成（最多 2 分钟）
      for (;;) {
        const result = await state.deps.runHeartbeatOnce({ reason: `cron:${job.id}` });
        if (result.status !== "skipped" || result.reason !== "requests-in-flight") brea
        if (state.deps.nowMs() - waitStartedAt > 2 * 60_000) {
          state.deps.requestHeartbeatNow({ reason: `cron:${job.id}` });
          return { status: "ok", summary: text };
        }
        await delay(250);
      }
    } else {
      // 异步模式：仅请求心跳，不等待
      state.deps.requestHeartbeatNow({ reason: `cron:${job.id}` });
    }
    return { status: "ok", summary: text };
  }

  // 模式二：隔离会话执行
  if (job.payload.kind !== "agentTurn") {
    return { status: "skipped", error: "isolated job requires agentTurn" };
  }
  const res = await state.deps.runIsolatedAgentJob({
    job,
    message: job.payload.message,
  });

  // 将摘要回传到主会话（如果配置了投递）
  const deliveryPlan = resolveCronDeliveryPlan(job);
  if (res.summary?.trim() && deliveryPlan.requested) {
    state.deps.enqueueSystemEvent(`Cron: ${res.summary}`, { agentId: job.agentId });
```

```
    if (job.wakeMode === "now") {
      state.deps.requestHeartbeatNow({ reason: `cron:${job.id}` });
    }
  }

  return { status: res.status, error: res.error, summary: res.summary };
}
```

> **衍生解释——心跳（Heartbeat）**：在 OpenClaw 中，心跳是 Agent 的"处理循环"。Agent 不是一直运行的——它在收到消息或系统事件时被唤醒，处理完后回到休眠状态。 `requestHeartbeatNow` 请求 Agent 立即唤醒并处理队列中的事件， `runHeartbeatOnce` 则同步等待一次心跳完成。Cron 的主会话模式通过注入系统事件 + 触发心跳来让 Agent "看到"定时消息。

隔离 Agent 的运行细节

`runCronIsolatedAgentTurn` （位于 `src/cron/isolated-agent/run.ts` ）是隔离执行的核心函数。它完成以下工作：

1. **会话管理**——为每次 Cron 运行创建或复用会话，会话键格式为 `agent:{agentId}:cron:{jobId}`
2. **模型选择**——优先使用作业指定的模型，回退到 Agent 默认模型
3. **思考级别**——支持 `thinking` 覆盖（low/medium/high/xhigh），xhigh 不支持的模型自动降级
4. **安全包装**——外部钩子（如 Gmail）的内容会被安全包装，防止提示注入
5. **投递处理**——Agent 完成后，根据投递计划将结果发送到指定通道

```
// src/cron/isolated-agent/run.ts — 会话键和标签（简化版）

const baseSessionKey = params.sessionKey || `cron:${params.job.id}`;
const runSessionKey = baseSessionKey.startsWith("cron:")
  ? `${agentSessionKey}:run:${runSessionId}`
  : baseSessionKey;

// 自动生成会话标签
if (!sessionEntry.label && baseSessionKey.startsWith("cron:")) {
  sessionEntry.label = `Cron: ${job.name || job.payload.message?.slice(0, 50)}`;
}
```

作业生命周期

完整的作业执行生命周期：

```
armTimer() → setTimeout() → onTimer() → locked() → findDueJobs()
    │
    ├── 标记 runningAtMs（防止重入）
    ├── emit("started")
    │
    ├── executeJobCore()
    │   ├── main → enqueueSystemEvent + requestHeartbeat
    │   └── isolated → runIsolatedAgentJob + 投递摘要
    │
    ├── 更新状态：lastRunAtMs, lastStatus, lastDurationMs
    ├── 计算 nextRunAtMs
    │   ├── at 类型 + ok → 禁用或删除
    │   └── every/cron 类型 → 计算下次时间
    │
    ├── emit("finished")
    ├── persist() → 写入磁盘
    └── armTimer() → 重新设置定时器
```

关键状态转换：

- **`at` 类型成功后**：如果 `deleteAfterRun=true` 则删除作业，否则禁用（`enabled=false`）
- **`every` / `cron` 类型**：基于结束时间计算下次运行时间，重新 arm 定时器
- **失败的作业**：记录 `lastError`，正常计算下次运行时间（不会因为失败就停止调度）

26.2.2 投递计划（Delivery Plan）（`src/cron/delivery.ts`）

隔离 Cron 作业完成后，Agent 的回复文本需要**投递**到用户——例如发送到 WhatsApp、Telegram 或者仅仅注入到主会话。投递计划（Delivery Plan）决定了投递的目标和方式。

投递配置

```
// src/cron/types.ts

export type CronDelivery = {
  mode: CronDeliveryMode;     // "announce" | "none"
  channel?: CronMessageChannel; // 通道 ID（如 "whatsapp"、"telegram"）或 "last"
  to?: string;                  // 目标收件人
  bestEffort?: boolean;         // 投递失败是否静默忽略
};
```

mode	行为
"announce"	将 Agent 回复投递到指定通道
"none"	不投递（仅记录运行结果）

投递计划解析

`resolveCronDeliveryPlan` 处理新旧两种配置格式：

```typescript
// src/cron/delivery.ts — resolveCronDeliveryPlan

export function resolveCronDeliveryPlan(job: CronJob): CronDeliveryPlan {
  const payload = job.payload.kind === "agentTurn" ? job.payload : null;
  const delivery = job.delivery;

  // 新格式：delivery 对象
  if (delivery) {
    const mode = delivery.mode ?? "announce";
    const channel = delivery.channel ?? payload?.channel ?? "last";
    const to = delivery.to ?? payload?.to;
    return {
      mode,
      channel,
      to,
      source: "delivery",
      requested: mode === "announce",
    };
  }

  // 旧格式：payload 内的 deliver/channel/to 字段
  const legacyMode = payload?.deliver === true
    ? "explicit"
    : payload?.deliver === false ? "off" : "auto";
  const hasExplicitTarget = Boolean(to);
  const requested = legacyMode === "explicit"
    || (legacyMode === "auto" && hasExplicitTarget);

  return {
    mode: requested ? "announce" : "none",
    channel,
    to,
    source: "payload",
    requested,
  };
}
```

`channel: "last"` 是一个特殊值——表示"使用用户最近活跃的消息通道"。例如用户最近在 WhatsApp 上和 Agent 对话，那么 Cron 的投递就会发到 WhatsApp。

26.2.3 运行日志（ src/cron/run-log.ts ）

每次 Cron 作业执行都会记录到一个 JSONL 格式的运行日志中。

日志文件组织

```
~/.openclaw/cron/
├── jobs.json          # 作业定义
└── runs/
    ├── job-abc.jsonl   # 作业 abc 的运行日志
    └── job-def.jsonl   # 作业 def 的运行日志
```

路径解析：

```
// src/cron/run-log.ts — resolveCronRunLogPath

export function resolveCronRunLogPath(params: { storePath: string; jobId: string }) {
  const dir = path.dirname(params.storePath);  // jobs.json 所在目录
  return path.join(dir, "runs", `${params.jobId}.jsonl`);
}
```

日志条目

```
export type CronRunLogEntry = {
  ts: number;           // 记录时间戳
  jobId: string;        // 作业 ID
  action: "finished";   // 动作类型（目前只有 finished）
  status?: "ok" | "error" | "skipped";  // 执行状态
  error?: string;       // 错误信息
  summary?: string;     // Agent 回复摘要
  sessionId?: string;   // 隔离会话 ID
  sessionKey?: string;  // 会话键
  runAtMs?: number;     // 实际开始时间
  durationMs?: number;  // 执行耗时
  nextRunAtMs?: number; // 下次计划运行时间
};
```

写入：顺序化 + 自动裁剪

```
// src/cron/run-log.ts — appendCronRunLog

const writesByPath = new Map<string, Promise<void>>();

export async function appendCronRunLog(filePath: string, entry: CronRunLogEntry, opts?)
  const resolved = path.resolve(filePath);
  // 顺序化：同一个文件的写入排队执行
  const prev = writesByPath.get(resolved) ?? Promise.resolve();
  const next = prev.catch(() => undefined).then(async () => {
    await fs.mkdir(path.dirname(resolved), { recursive: true });
    await fs.appendFile(resolved, `${JSON.stringify(entry)}\n`, "utf-8");

    // 自动裁剪：超过 2MB 时保留最近 2000 行
    await pruneIfNeeded(resolved, {
      maxBytes: opts?.maxBytes ?? 2_000_000,
      keepLines: opts?.keepLines ?? 2_000,
    });
  });
  writesByPath.set(resolved, next);
  await next;
}
```

自动裁剪防止日志文件无限增长。当文件超过 2MB 时，只保留最近 2000 行。裁剪通过"写入临时文件 → 原子重命名"实现，避免裁剪过程中的数据丢失。

读取：倒序扫描

```
// src/cron/run-log.ts — readCronRunLogEntries

export async function readCronRunLogEntries(filePath, opts?): Promise<CronRunLogEntry[]
  const limit = Math.min(5000, opts?.limit ?? 200);
  const raw = await fs.readFile(filePath, "utf-8").catch(() => "");
  const parsed: CronRunLogEntry[] = [];
  const lines = raw.split("\n");

  // 从末尾开始扫描（最新的在前）
  for (let i = lines.length - 1; i >= 0 && parsed.length < limit; i--) {
    const line = lines[i]?.trim();
    if (!line) continue;
    const obj = JSON.parse(line);
    if (obj.action !== "finished") continue;  // 只读取 finished 条目
    parsed.push(entry);
  }

  return parsed.toReversed();  // 返回时恢复正序
}
```

从末尾扫描确保了即使文件很大，也只需要读取最近的 N 条记录。

26.2.4 Cron 存储与迁移（`src/cron/store.ts`）

存储格式

所有 Cron 作业存储在一个 JSON 文件中：

```json
{
  "version": 1,
  "jobs": [
    {
      "id": "abc-123",
      "name": "每日新闻摘要",
      "enabled": true,
      "schedule": { "kind": "cron", "expr": "0 8 * * *", "tz": "Asia/Shanghai" },
      "sessionTarget": "isolated",
      "wakeMode": "now",
      "payload": { "kind": "agentTurn", "message": "请搜索今天的科技新闻并生成摘要" },
      "delivery": { "mode": "announce", "channel": "whatsapp" },
      "state": { "nextRunAtMs": 1709280000000, "lastStatus": "ok" },
      "createdAtMs": 1709000000000,
      "updatedAtMs": 1709200000000
    }
  ]
}
```

存储路径

```typescript
// src/cron/store.ts

export const DEFAULT_CRON_DIR = path.join(CONFIG_DIR, "cron");
export const DEFAULT_CRON_STORE_PATH = path.join(DEFAULT_CRON_DIR, "jobs.json");

export function resolveCronStorePath(storePath?: string) {
  if (storePath?.trim()) {
    const raw = storePath.trim();
    if (raw.startsWith("~")) {
      return path.resolve(raw.replace("~", os.homedir()));
    }
    return path.resolve(raw);
  }
  return DEFAULT_CRON_STORE_PATH;  // ~/.openclaw/cron/jobs.json
}
```

加载与保存

```
// src/cron/store.ts — loadCronStore

export async function loadCronStore(storePath: string): Promise<CronStoreFile> {
  try {
    const raw = await fs.promises.readFile(storePath, "utf-8");
    const parsed = JSON5.parse(raw);  // 支持 JSON5 格式（允许注释和尾逗号）
    const jobs = Array.isArray(parsed.jobs) ? parsed.jobs.filter(Boolean) : [];
    return { version: 1, jobs };
  } catch (err) {
    if (err.code === "ENOENT") {
      return { version: 1, jobs: [] };  // 文件不存在→空存储
    }
    throw err;
  }
}
```

> **衍生解释——JSON5**：JSON5 是 JSON 的超集，允许注释（`//` 和 `/* */`）、尾逗号、单引号字符串、多行字符串等。OpenClaw 使用 JSON5 解析 Cron 存储，允许用户手动编辑 `jobs.json` 文件时使用更友好的语法。

```
// src/cron/store.ts — saveCronStore

export async function saveCronStore(storePath: string, store: CronStoreFile) {
  await fs.promises.mkdir(path.dirname(storePath), { recursive: true });

  // 原子写入：先写临时文件，再重命名
  const tmp = `${storePath}.${process.pid}.${Math.random().toString(16).slice(2)}.tmp`;
  await fs.promises.writeFile(tmp, JSON.stringify(store, null, 2), "utf-8");
  await fs.promises.rename(tmp, storePath);

  // 最佳努力备份
  try {
    await fs.promises.copyFile(storePath, `${storePath}.bak`);
  } catch { /* ignore */ }
}
```

原子写入通过"写入临时文件 → `rename`"实现。`rename` 在同一文件系统上是原子操作，确保即使写入中途进程崩溃，也不会损坏原始文件。每次保存后还会创建一个 `.bak` 备份。

Gateway 集成

Cron 服务在 Gateway 启动时初始化：

```
// src/gateway/server-cron.ts — buildGatewayCronService（简化版）

export function buildGatewayCronService(params) {
  const storePath = resolveCronStorePath(params.cfg.cron?.store);
  const cronEnabled = process.env.OPENCLAW_SKIP_CRON !== "1"
    && params.cfg.cron?.enabled !== false;

  const cron = new CronService({
    storePath,
    cronEnabled,
    log: getChildLogger({ module: "cron" }),
    enqueueSystemEvent: params.enqueueSystemEvent,
    requestHeartbeatNow: params.requestHeartbeatNow,
    runHeartbeatOnce: params.runHeartbeatOnce,
    runIsolatedAgentJob: async ({ job, message }) => {
      return await runCronIsolatedAgentTurn({
        job,
        message,
        sessionKey: `cron:${job.id}`,
        lane: "cron",
        cfg: params.cfg,
        // ... 更多参数
      });
    },
    onEvent: (evt) => {
      params.broadcast("cron", evt, { dropIfSlow: true });
      // 运行完成后追加运行日志
      if (evt.action === "finished") {
        appendCronRunLog(logPath, { ...evt, ts: Date.now() });
      }
    },
  });

  return { cron, storePath, cronEnabled };
}
```

关键配置：

- **`OPENCLAW_SKIP_CRON=1`** ——环境变量可以完全禁用 Cron（不加载、不调度）

- **`cron.enabled: false`** ——配置文件禁用（作业保存但不调度）
- **`onEvent` 广播**——每次作业状态变更都会通过 WebSocket 广播到所有连接的客户端
- **`lane: "cron"`** ——Cron 作业运行在专用的命令车道上，与主会话隔离

> **v2026.3.9 Breaking Change：Cron 隔离强化**
>
> 自 v2026.3.9 起，Cron 作业**不再支持**通过临时 Agent 发送（ad hoc agent sends）或主会话回退摘要（fallback main-session summaries）进行通知。所有 Cron 通知必须通过正式的**投递计划**（Delivery Plan，参见上文 18.2.2 节）配置。升级时使用 `openclaw doctor --fix` 可自动处理旧版 Cron 存储和 notify/webhook 元数据的迁移。这一变更加强了 Cron 作业的隔离性——确保定时任务不会意外干扰主会话的对话流。

本节小结

1. Cron 作业有两种执行模式：**主会话**（注入系统事件 + 触发心跳）和**隔离会话**（独立 Agent 运行）
2. 隔离 Agent 执行涉及会话管理、模型选择、思考级别控制、安全包装和投递处理
3. **投递计划**（Delivery Plan）决定隔离作业结果的去向，支持新版 `delivery` 对象和旧版 `payload` 字段两种配置
4. 运行日志使用 **JSONL 格式**，每个作业一个文件，写入顺序化防止竞争，自动裁剪防止无限增长
5. 作业存储使用 **JSON5 文件**（`~/.openclaw/cron/jobs.json`），支持手动编辑
6. 存储操作通过**原子写入**（临时文件 + rename）保障数据安全，每次保存自动创建 `.bak` 备份
7. Gateway 集成时将 Cron 事件通过 WebSocket **广播**给客户端，并追加运行日志